

# 배포

- Docker & GHCR &  
AWS EC2

# 도커 설치하기

- WSL2 설치(Windows PowerShell에서 실행) 여부 확인

- `wsl --status`
- `wsl.exe --install`

```
user@LAPTOP-6FOGKB6J: /mnt/c/Windows/system32
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

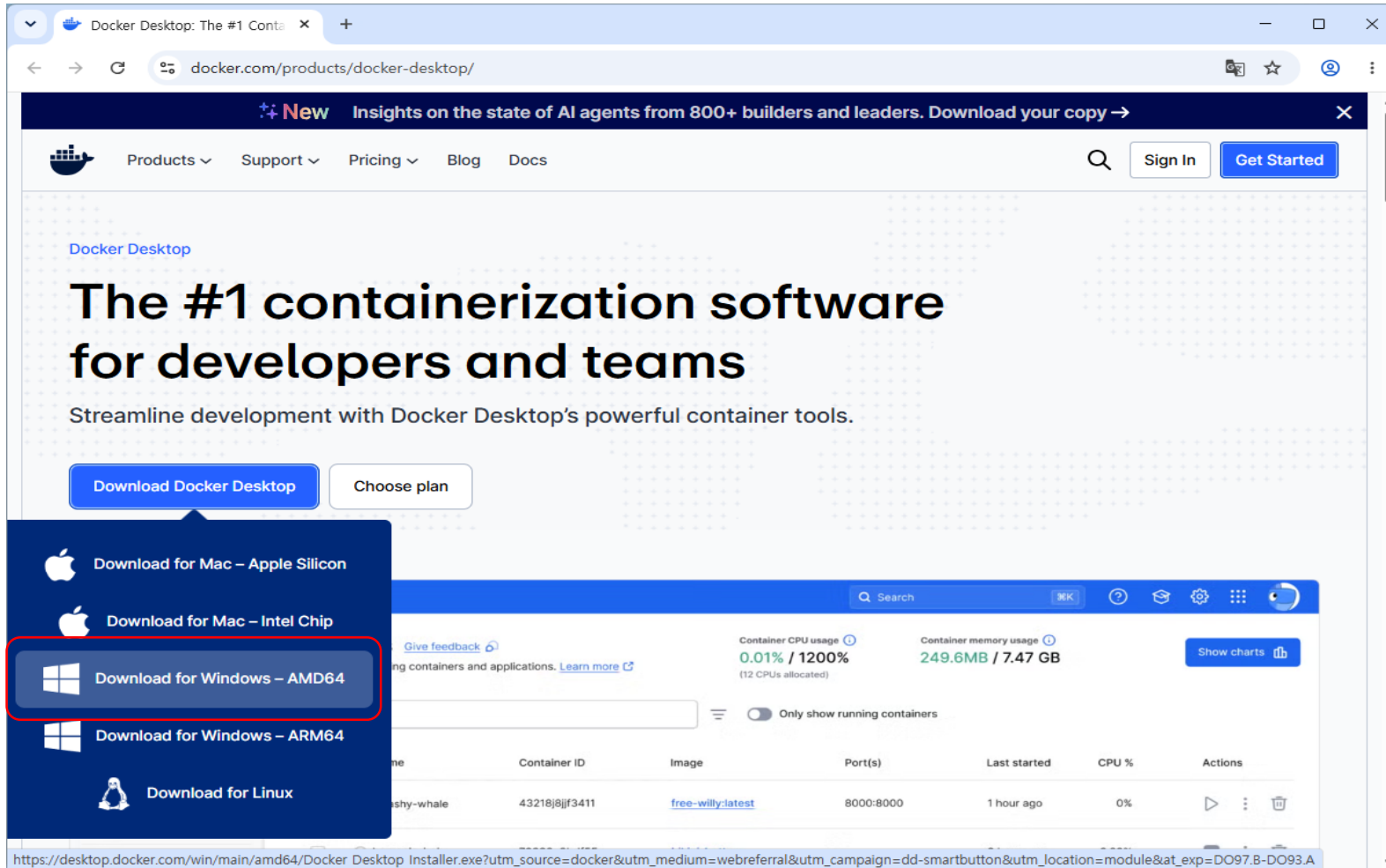
새로운 기능 및 개선 사항에 대 한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows

PS C:\Windows\system32> wsl --status
Linux용 Windows 하위 시스템 설치되어 있지 않습니다. 'wsl.exe --install'을 실행하여 설치할 수 있습니다.
자세한 내용은 https://aka.ms/wslinstall
참조하세요.
PS C:\Windows\system32> wsl.exe --install
다운로드 중: Linux용 Windows 하위 시스템 2.7.8
설치 중: Linux용 Windows 하위 시스템 2.7.8
Linux용 Windows 하위 시스템 2.7.8이(가) 설치되었습니다.
작업을 완료했습니다.
다운로드 중: Ubuntu
설치 중: Ubuntu
배포가 설치되었습니다. 'wsl.exe -d Ubuntu'을(를) 통해 시작할 수 있습니다.
Ubuntu 시작하는 중...
Provisioning the new WSL instance Ubuntu
This might take a while...
```

# 도커 설치하기

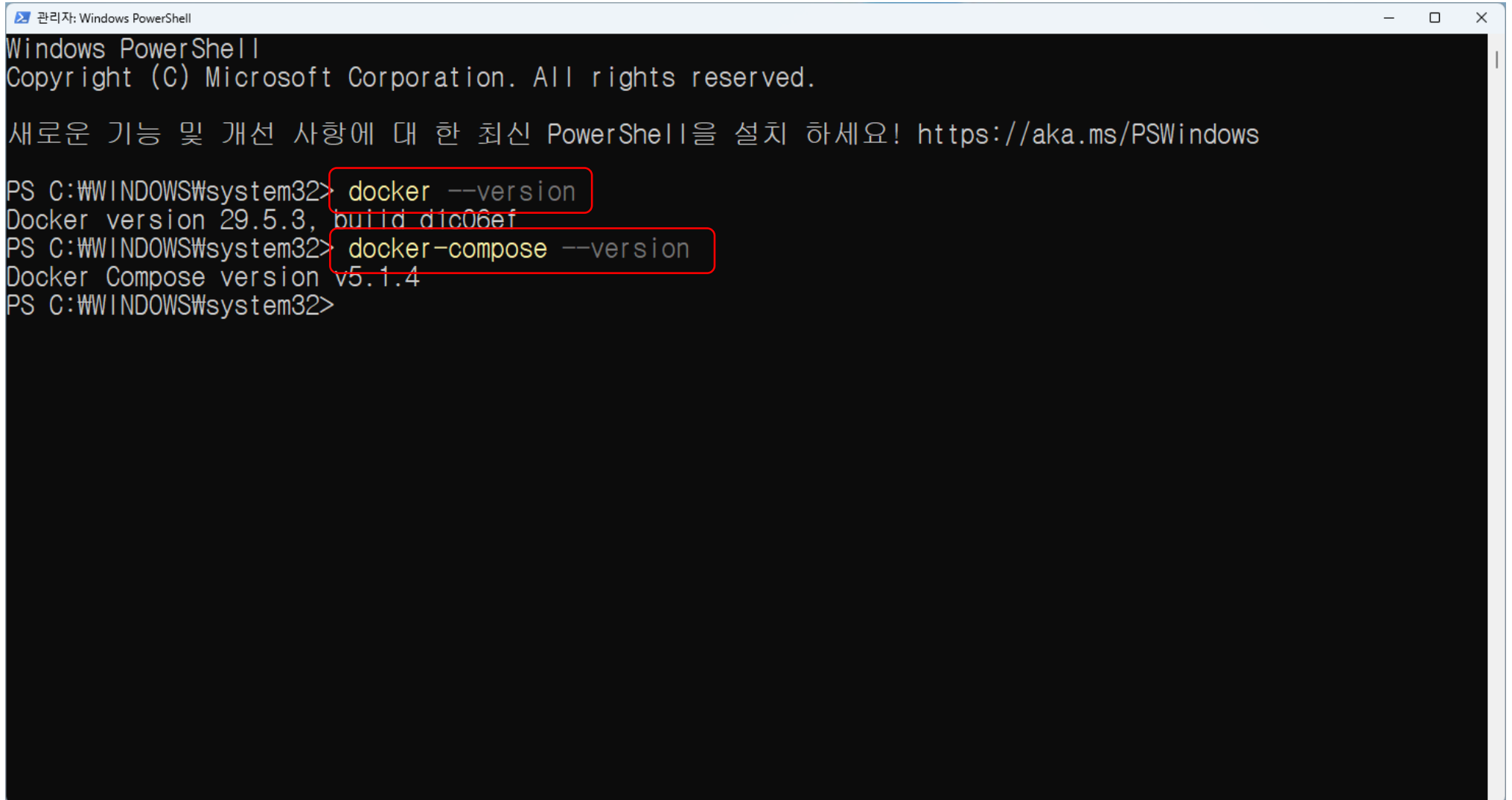
- 도커 데스크탑 웹 사이트

- <https://docker.com/products/docker-desktop>



# 도커 설치하기

- 설치 확인



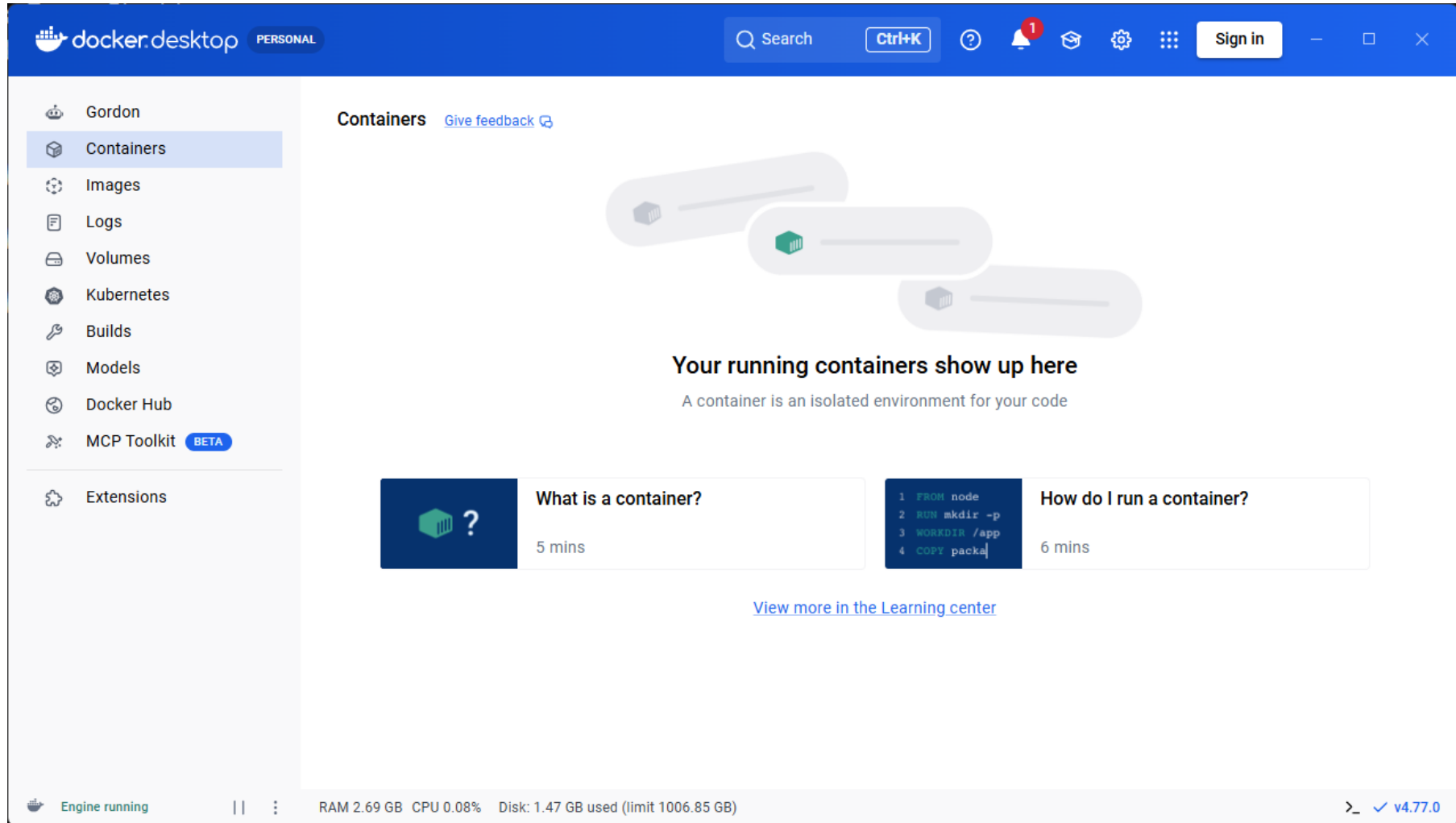
```
관리자: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> docker --version
Docker version 29.5.3, build d1c06ef
PS C:\WINDOWS\system32> docker-compose --version
Docker Compose version v5.1.4
PS C:\WINDOWS\system32>
```

# 도커 설치하기

## ● 실행 화면



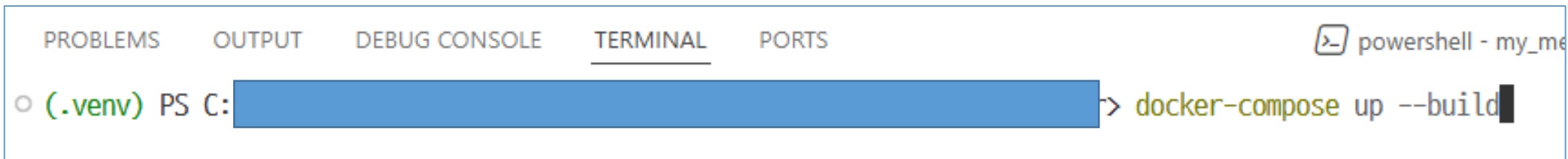
# 도커로 시스템 실행하기

- 기본 프로젝트 생성하기

- docker-compose.yml 작성
- requirements.txt 추가
- backend/ Dockerfile 작성

- 기본 프로젝트 실행하기

```
docker-compose up --build
```

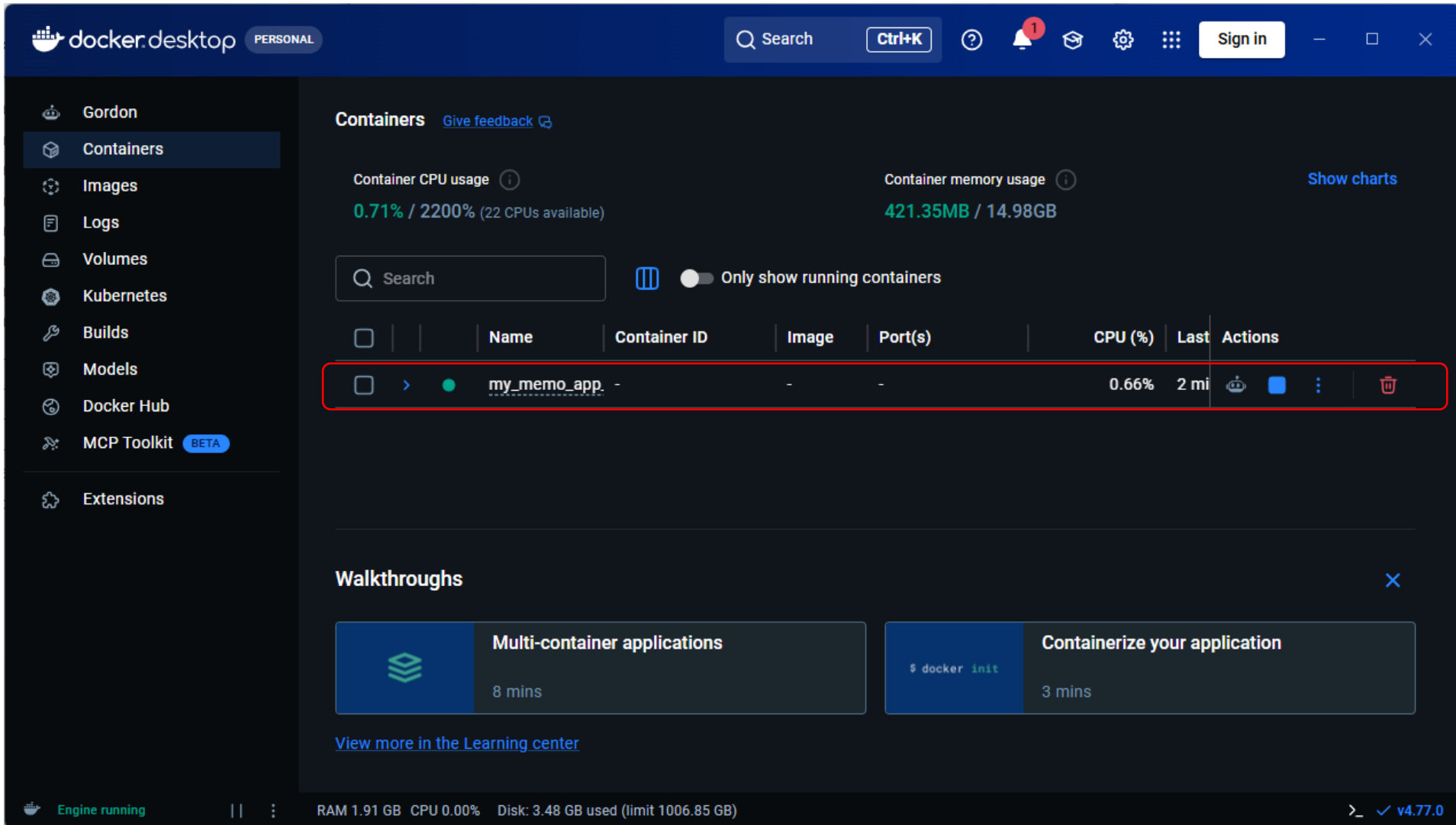


The screenshot shows a VS Code terminal window with the following elements:

- Tab bar: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS
- Terminal title: powershell - my\_me
- Terminal content: (.venv) PS C: [blue selection bar] -> docker-compose up --build

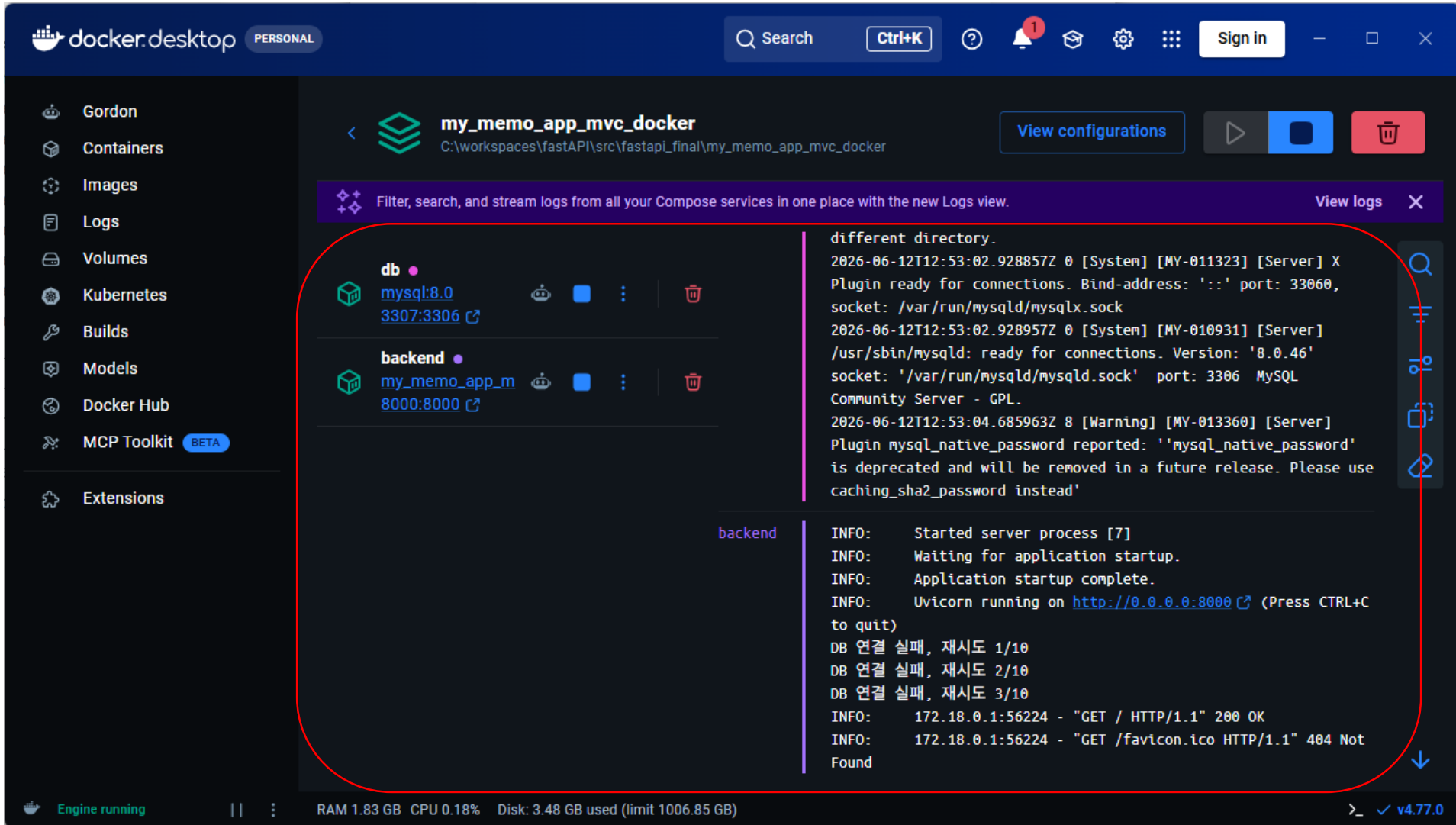
# 도커로 시스템 실행하기

- 도커 데스크탑에서 실행 확인



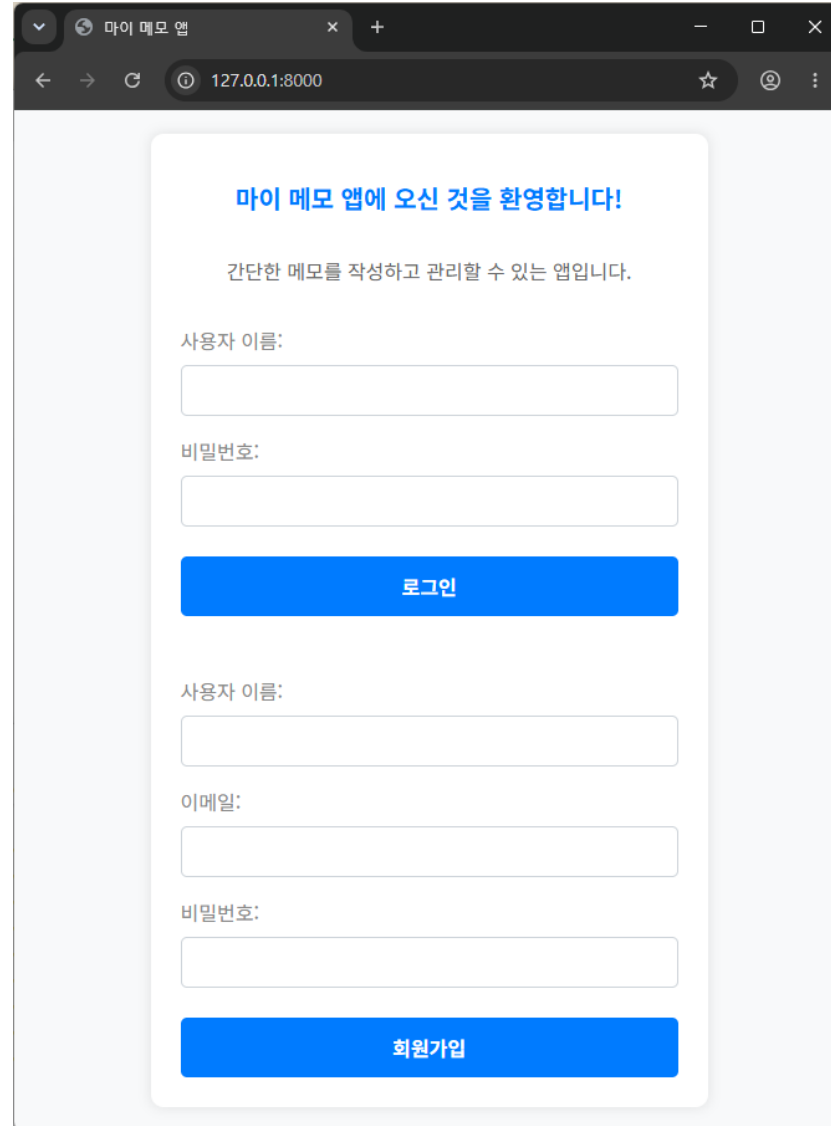
# 도커로 시스템 실행하기

- 도커 데스크탑에서 실행 확인



# 애플리케이션에 접속하기

- <http://127.0.0.1:8000/>



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000". The page content is as follows:

**마이 메모 앱에 오신 것을 환영합니다!**

간단한 메모를 작성하고 관리할 수 있는 앱입니다.

사용자 이름:

비밀번호:

**로그인**

사용자 이름:

이메일:

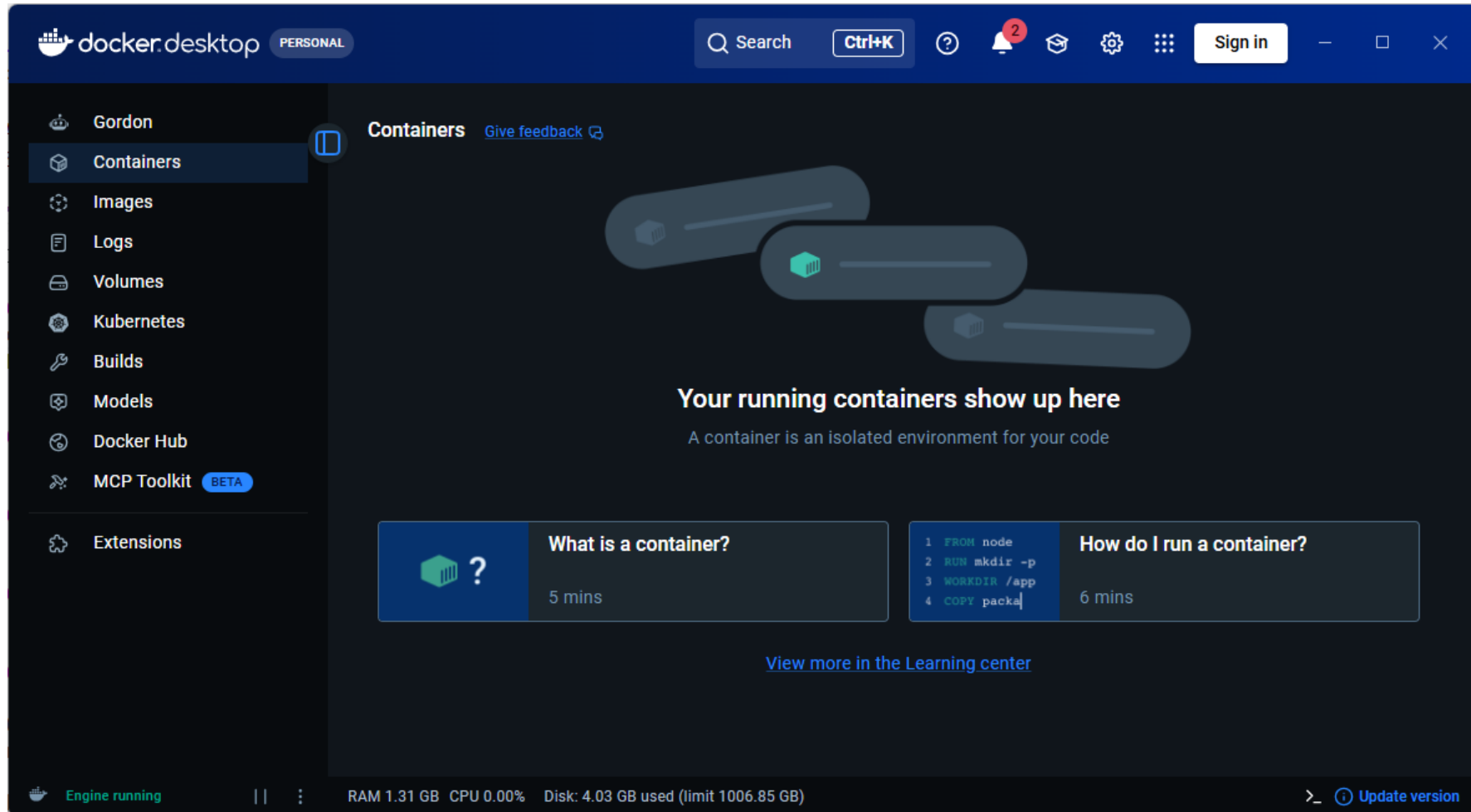
비밀번호:

**회원가입**

# 서비스 종료하기

- 애플리케이션 종료를 위해 터미널에서 [Ctrl]+[C] 키를 누른 후, 아래 명령어 실행

**docker-compose down**



# 클라우드 배포를 위한 시스템 구성하기

---

- 프로젝트를 로컬에서 도커 이미지를 만들기 위해 먼저 빌드
  - 이 이미지를 깃허브의 **컨테이너 레지스트리**(GitHub Container Registry, GHCR)로 푸시(push)
  - **AWS EC2**에서는 GHCR에 푸시된 백엔드의 도커 이미지를 이용해서 백엔드, 데이터베이스를 실행시킬 것
- 
- **컨테이너 레지스트리**
    - 도커 이미지를 클라우드에 배포하기 위한 '중간 저장소'
- 
- **AWS EC2**
    - 클라우드에서 실행되는 '가상 서버'
    - 컨테이너 레지스트리에 배포된 도커 이미지를 가져와서 실행하는 역할을 한다고 이해

# AWS EC2 인스턴스 준비하기

## ● AWS EC2 인스턴스 준비하기

- <https://aws.amazon.com/ko/>
- 회원 가입 및 로그인

[illegible]

# AWS EC2 인스턴스 준비하기

- 새로운 EC2 인스턴스 생성 화면

**인스턴스 시작** 정보

Amazon EC2를 사용하면 AWS 클라우드에서 실행되는 가상 머신 또는 인스턴스를 생성할 수 있습니다. 아래의 간단한 단계에 따라 빠르게 시작할 수 있습니다.

이름 및 태그 정보

이름

fastAPI

추가 태그 추가

# AWS EC2 인스턴스 준비하기

- 애플리케이션 및 OS 이미지

▼ 애플리케이션 및 OS 이미지(Amazon Machine Image) 정보

AMI에는 인스턴스의 운영 체제, 애플리케이션 서버, 애플리케이션이 포함됩니다. 아래에 적합한 AMI가 보이지 않는 경우 검색 필드를 사용하거나 더 많은 AMI 찾아보기를 선택하세요.

🔍 수천 개의 애플리케이션 및 OS 이미지를 포함하는 전체 카탈로그 검색

최근 사용

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

Debian

debian

🔍

더 많은 AMI 찾아보기

AWS, Marketplace 및 커뮤니티의 AMI 포함

Amazon Machine Image(AMI)

Amazon Linux 2023 kernel-6.1 AMI  
ami-0152204c1a187337c (64비트(x86), uefi-preferred) / ami-02c114835b4e7739f (64비트(Arm), uefi)  
가상화: hvm   ENA 활성화됨: true   루트 디바이스 유형: ebs

프리 티어 사용 가능 ▼

설명

Amazon Linux 2023(kernel-6.1)은 5년의 장기 지원을 제공하는 최신 범용 Linux 기반 OS입니다. AWS에 최적화되어 있으며 클라우드 애플리케이션을 개발 및 실행할 수 있는 안전하고 안정적인 고성능 실행 환경을 제공하도록 설계되었습니다.

Amazon Linux 2023 AMI 2023.12.20260608.0 x86\_64 HVM kernel-6.1

아키텍처

부트 모드

AMI ID

게시 날짜

사용자 이름 ⓘ

확인된 공급 업체

64비트(x86) ▼

uefi-preferred

ami-██████████

2026-06-04

ec2-user

# AWS EC2 인스턴스 준비하기

- 인스턴스 유형

- t2.micro 선택

▼ 인스턴스 유형 정보 | 조언 받기

인스턴스 유형

t2.micro

패밀리: t2 1 vCPU 1 GiB 메모리 현재 세대: true 온디맨드 Linux 기본 요금: 0.0116 USD 시간당  
온디맨드 Windows 기본 요금: 0.0162 USD 시간당 온디맨드 Ubuntu Pro 기본 요금: 0.0134 USD 시간당  
온디맨드 SUSE 기본 요금: 0.0116 USD 시간당 온디맨드 RHEL 기본 요금: 0.026 USD 시간당

☐ 모든 세대

인스턴스 유형 비교

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

# AWS EC2 인스턴스 준비하기

- 키 페어 생성 - 생성된 파일 잘 보관할 것

▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - 필수

키 페어 생성

키 페어 이름

키 페어를 사용하면 인스턴스에 안전하게 연결할 수 있습니다.

키 페어 이름 입력

이름에는 최대 255개의 ASCII 문자가 포함됩니다. 앞 또는 뒤에 공백을 포함할 수 없습니다.

키 페어 유형

☒ RSA  
RSA 암호화된 프라이빗 및 퍼블릭 키 페어

☐ ED25519  
ED25519 암호화된 프라이빗 및 퍼블릭 키 페어

프라이빗 키 파일 형식

☒ .pem  
OpenSSH와 함께 사용

☐ .ppk  
PuTTY와 함께 사용

⚠ 메시지가 표시되면 프라이빗 키를 사용자 컴퓨터의 안전하고 액세스 가능한 위치에 저장합니다. 나중에 인스턴스에 연결할 때 필요합니다. 자세히 알아보기

취소

키 페어 생성

새 키 페어 생성

# AWS EC2 인스턴스 준비하기

- 나머지 옵션은 디폴트 그대로
- 인스턴스 시작

▼ 요약

인스턴스 개수 | 정보

1

소프트웨어 이미지(AMI)

Amazon Linux 2023 AMI 2023.12....[더 보기](#)  
ami-0152204c1a187337c

가상 서버 유형(인스턴스 유형)

t2.micro

방화벽(보안 그룹)

새 보안 그룹

스토리지(볼륨)

1개의 볼륨 – 8GiB

❗ 프리 티어: AWS 계정을 개설한 첫해에 프리 티어 AMI, 매월 750 시간의 퍼블릭 IPv4 주소 사용량, 30GiB의 EBS 스토리지, 2백만 I/O, 1GB의 스냅샷, 100GB의 인터넷 대역폭과 함께 사용하면 매월 750시간의 t2.micro 인스턴스 사용량(또는 t2.micro를 사용할 수 없는 경우 t3.micro)을 제공합니다. 데이터 전송 요금은 프리티어 허용 범위에 포함되지 않습니다. 계정의 프리 티어 상태에 따라 요금이 부과될 수 있습니다.

×

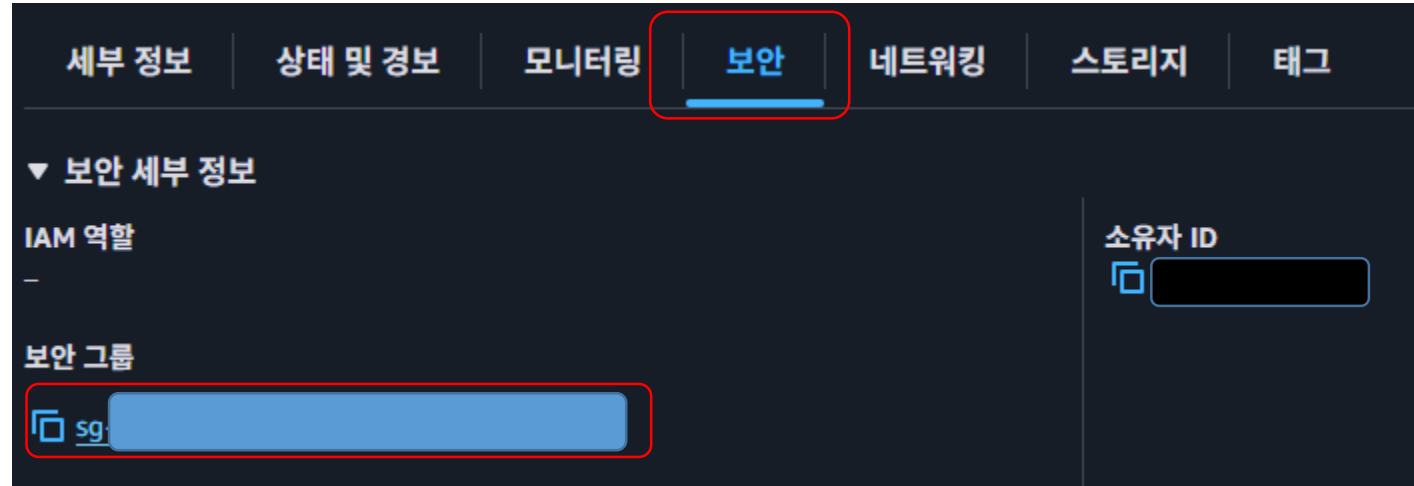
취소

인스턴스 시작

미리 보기 코드

# AWS EC2 인스턴스 준비하기

- 인바운드 규칙 편집



# AWS EC2 인스턴스 준비하기

- 인바운드 규칙 편집

### 인바운드 규칙 편집

인바운드 규칙은 인스턴스에 도달하도록 허용된 수신 트래픽을 제어합니다.

#### 인바운드 규칙

| 보안 그룹 규칙 ID | 유형         | 정보 | 프로토콜 | 포트 범위 | 소스   | 정보                   | 설명 - 선택 사항 | 정보 |
|-------------|------------|----|------|-------|------|----------------------|------------|----|
| sg-         | SSH        |    | TCP  | 22    | 사... | <div>0.0.0.0/0</div> |            | 삭제 |
| -           | HTTP       |    | TCP  | 80    | A... | <div>0.0.0.0/0</div> |            | 삭제 |
| -           | HTTPS      |    | TCP  | 443   | A... | <div>0.0.0.0/0</div> |            | 삭제 |
| -           | 사용자 지정 TCP |    | TCP  | 8000  | A... | <div>0.0.0.0/0</div> |            | 삭제 |

규칙 추가

⚠ 소스가 0.0.0.0/0 또는 ::/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다.

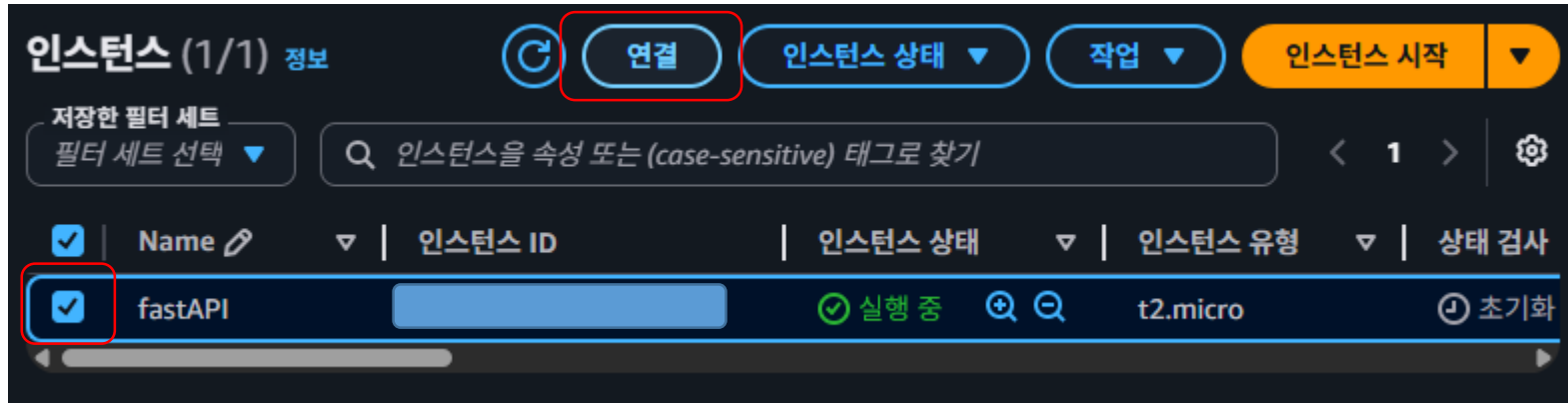
취소

변경 사항 미리 보기

규칙 저장

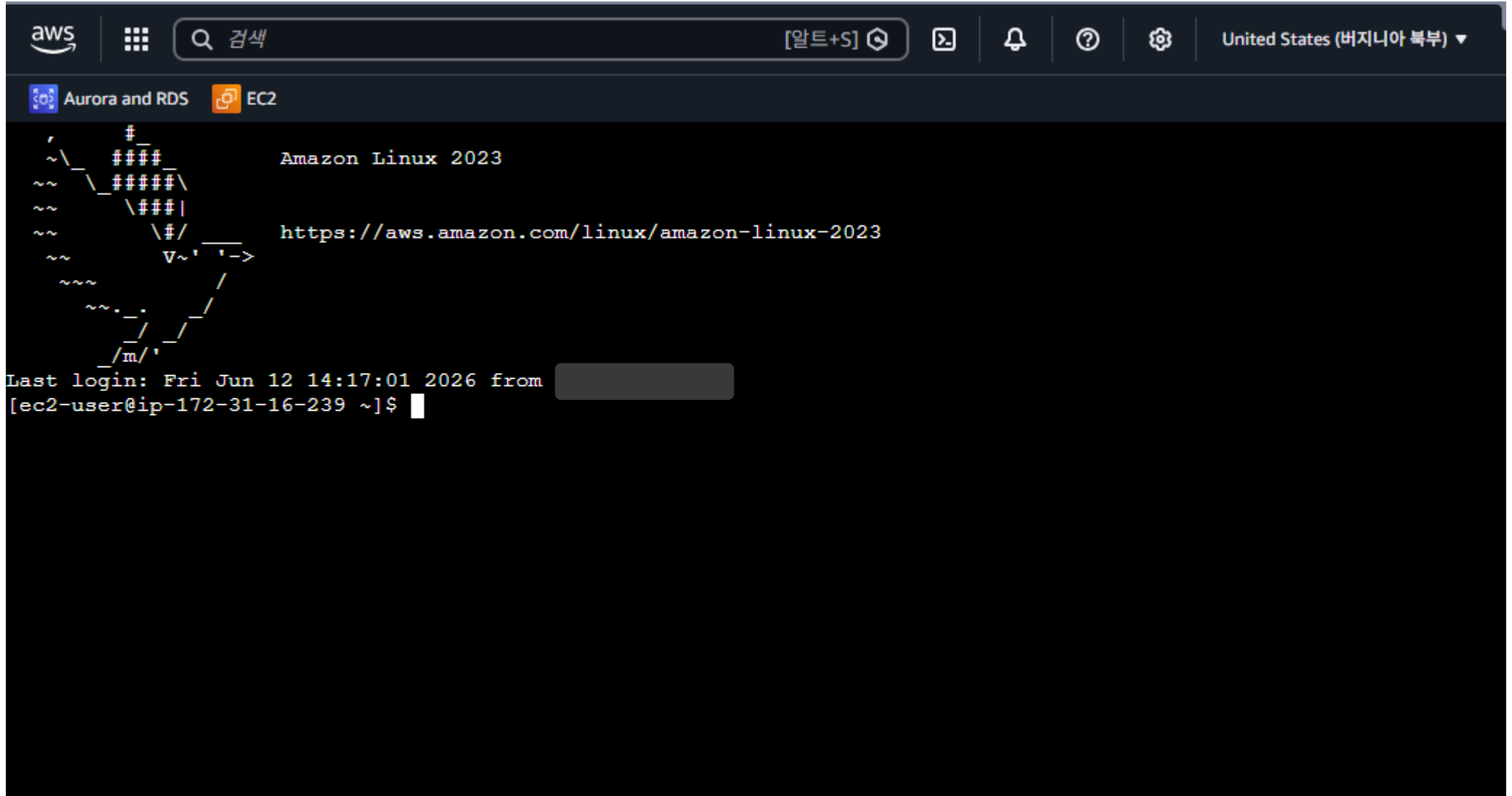
# AWS EC2 인스턴스 준비하기

- 인스턴스를 선택하고 연결하기



# AWS EC2 인스턴스 준비하기

- 인스턴스를 선택하고 연결하기



The screenshot displays the AWS Management Console interface. At the top, the AWS logo is on the left, followed by a search bar containing the text '검색'. To the right of the search bar are several icons: a keyboard shortcut '[알트+S]', a document icon, a notification bell, a help icon, and a settings gear. The region is set to 'United States (버지니아 북부)'. Below the navigation bar, the 'Aurora and RDS' and 'EC2' service tabs are visible. The main content area shows a terminal window for an Amazon Linux 2023 instance. The terminal output includes a ASCII art logo for Amazon Linux, the text 'Amazon Linux 2023', and a URL 'https://aws.amazon.com/linux/amazon-linux-2023'. At the bottom of the terminal, it shows the last login time 'Last login: Fri Jun 12 14:17:01 2026 from [redacted]' and the current prompt '[ec2-user@ip-172-31-16-239 ~]\$'.

```
aws | [검색] | [알트+S] | [문서] | [벨] | [?] | [설정] | United States (버지니아 북부) ▼  
Aurora and RDS | EC2  
#  
~\##### Amazon Linux 2023  
~~\#####  
~~\###|  
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~V~' '->  
~~~  
~~.-.  
~~/_/m/'  
Last login: Fri Jun 12 14:17:01 2026 from [redacted]  
[ec2-user@ip-172-31-16-239 ~]$
```

# AWS EC2 인스턴스 준비하기

- 파워셀에서 ssh 명령을 이용해서 EC2에 접속한 화면

[illegible]

# AWS EC2 인스턴스 준비하기

---

- Amazon Linux 2023에 Docker 설치
  - `sudo dnf update -y`
  - `sudo dnf install docker -y`
- Docker 서비스 시작
  - `sudo systemctl enable --now docker`
  - `sudo systemctl start docker`
- 현재 사용자에게 Docker 권한 부여
  - `sudo usermod -aG docker ec2-user`
- 한 번 로그아웃 후 다시 접속
  - `exit`
- 재접속 후
  - `docker --version`

# AWS EC2 인스턴스 준비하기

---

- Amazon Linux 2023에 Docker Compose v2 설치

- Compose 플러그인 설치

- 모든 사용자용 시스템 경로 생성

- `mkdir -p ~/.docker/cli-plugins`

- 최신 버전을 다운로드

- `curl -SL ₩`

`https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86\_64 ₩`

`-o ~/.docker/cli-plugins/docker-compose`

- 실행 권한 부여

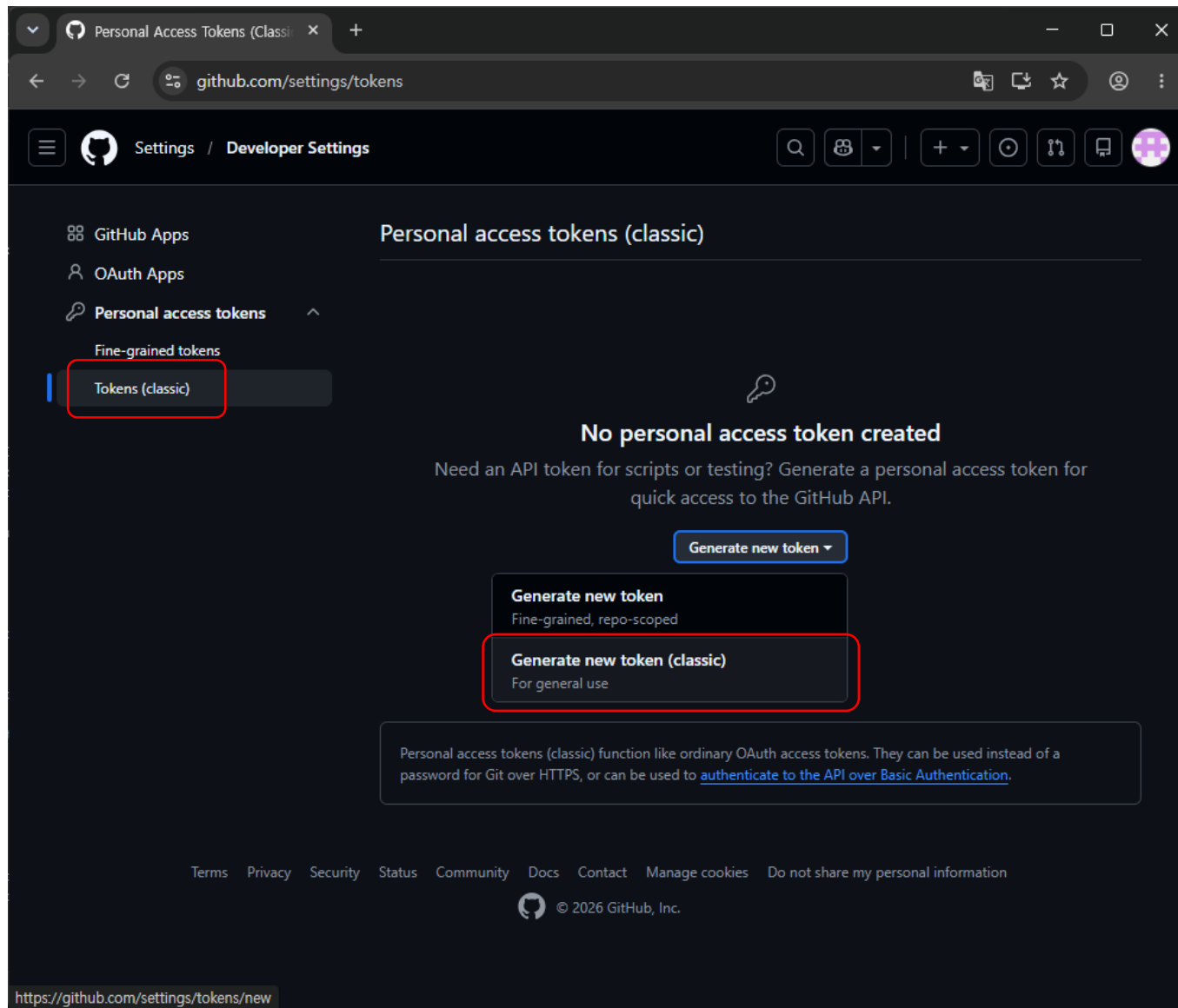
- `chmod +x ~/.docker/cli-plugins/docker-compose`

- 확인 - 이제 하이픈(-) 없이 docker compose 동작

- `docker compose version`

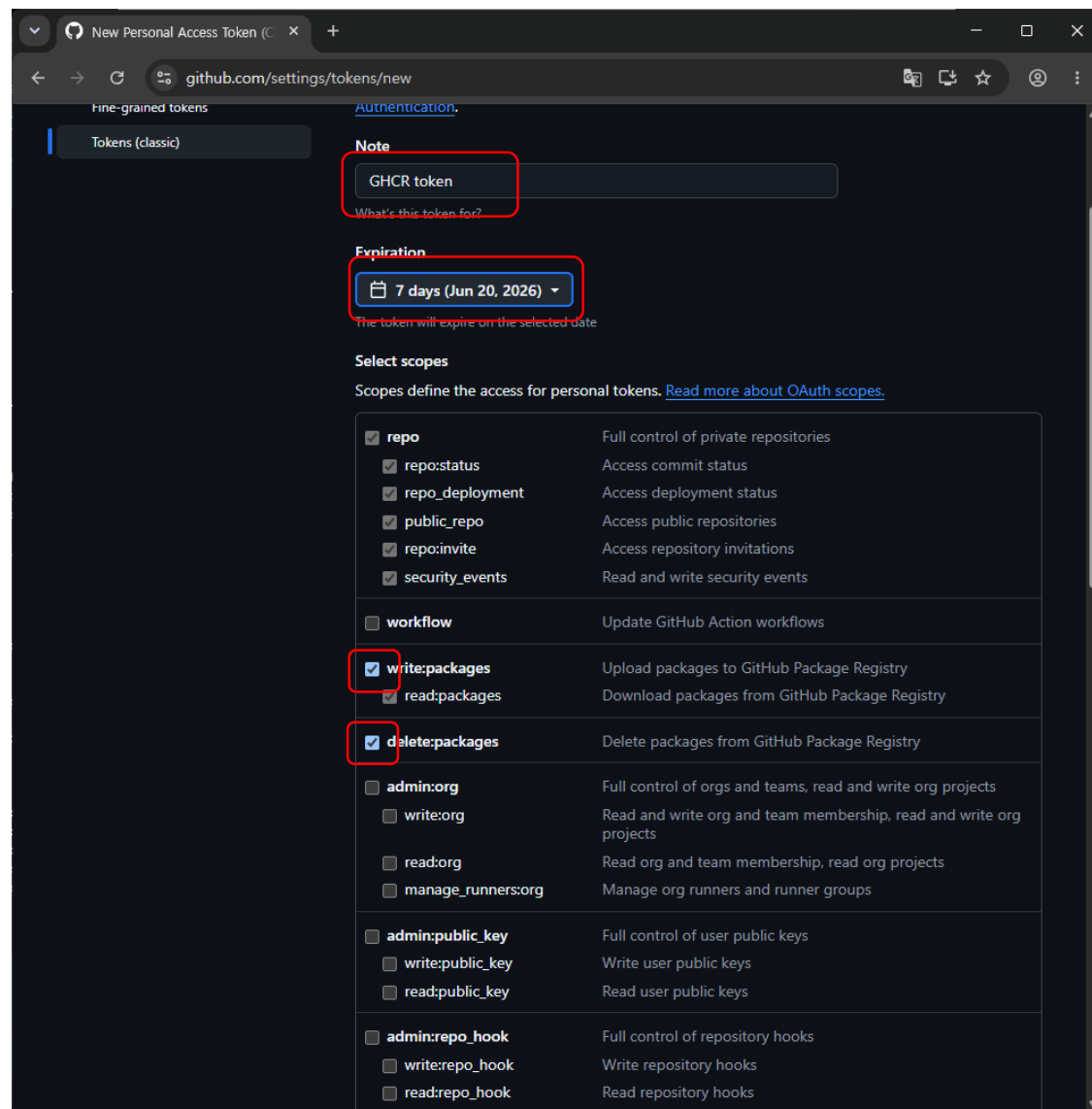
# 깃허브 컨테이너 레지스트리 설정하기

- 컨테이너 레지스트리를 이용하기 위한 전용 개인 액세스 토큰 발급



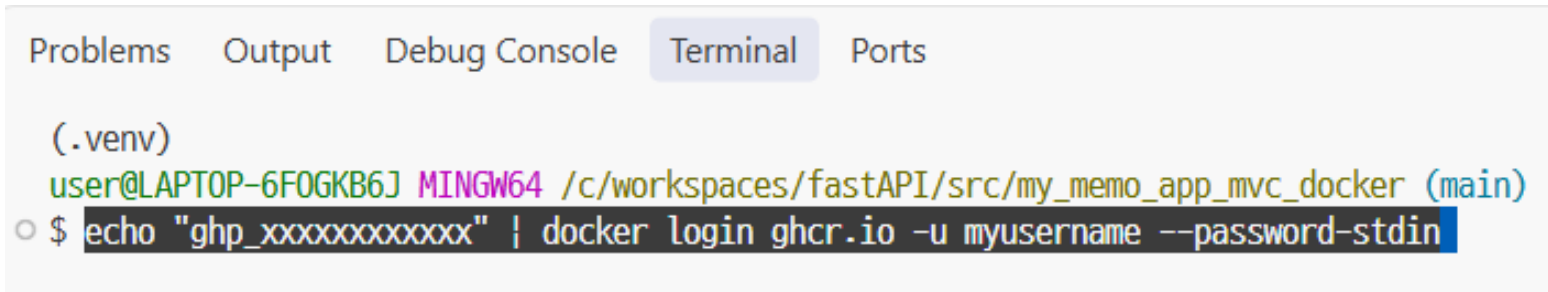
# 깃허브 컨테이너 레지스트리 설정하기

- 컨테이너 레지스트리를 이용하기 위한 전용 개인 액세스 토큰 발급



# AWS EC2 인스턴스 준비하기

- vs code 터미널에서 아래 명령 실행하여 GHCR 로그인
  - `echo $GHCR_TOKEN | docker login ghcr.io -u <GitHubID> --password-stdin`



The screenshot shows a VS Code interface with a terminal window open. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (which is active), and 'Ports'. The terminal output shows the prompt '(.venv)' followed by the user's shell prompt 'user@LAPTOP-6FOGKB6J MINGW64 /c/workspaces/fastAPI/src/my\_memo\_app\_mvc\_docker (main)'. The command being executed is 'echo "ghp\_XXXXXXXXXXXX" | docker login ghcr.io -u myusername --password-stdin', which is highlighted with a blue selection background.

```
Problems Output Debug Console Terminal Ports
(.venv)
user@LAPTOP-6FOGKB6J MINGW64 /c/workspaces/fastAPI/src/my_memo_app_mvc_docker (main)
$ echo "ghp_XXXXXXXXXXXX" | docker login ghcr.io -u myusername --password-stdin
```

Login Succeeded

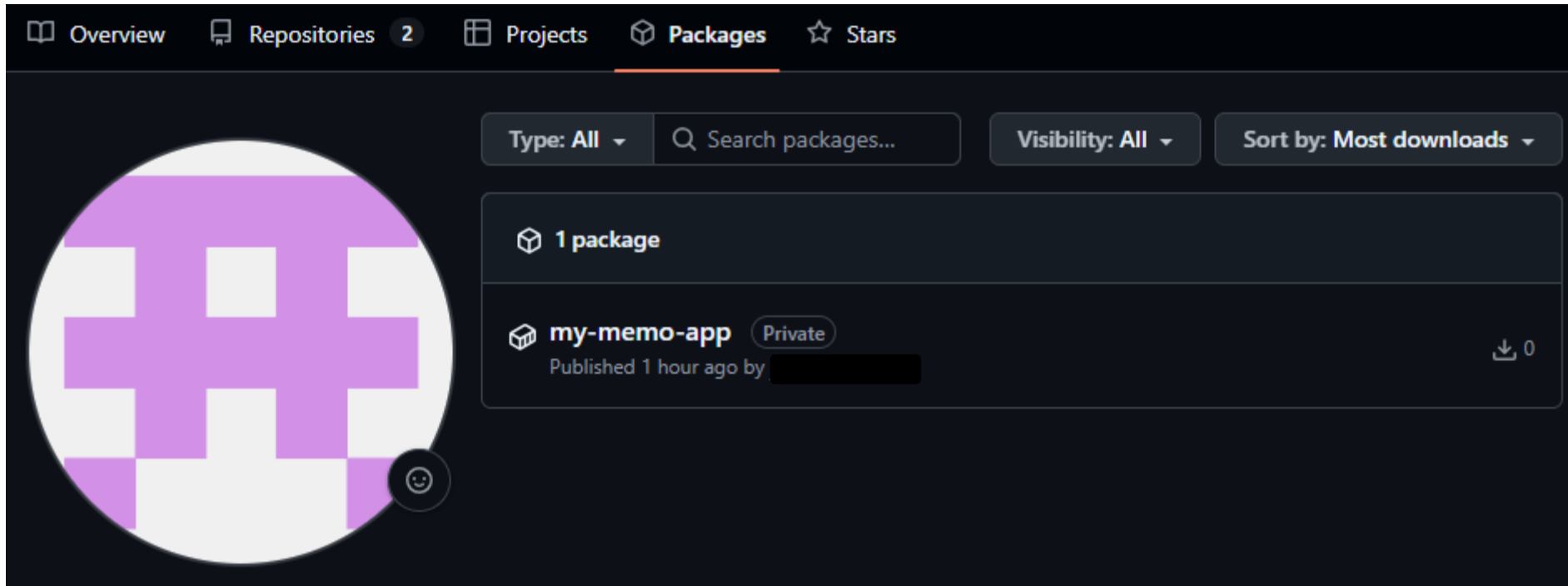
# GHCR을 이용한 도커 컨테이너 배포하기

---

- 깃허브 컨테이너 레지스트리에 접속(파워셸)
  - `docker login ghcr.io -u [깃랩 사용자 이름] -p [개인 액세스 토큰]`
- backend 폴더로 이동한 다음, backend 폴더에 있는 도커 파일을 이용해서 도커 이미지를 빌드(파워셸)
  - `docker build --platform linux/amd64 -t ghcr.io/[깃랩 사용자 이름]/[패키지 이름]:1.0.0 .`
- 도커 이미지가 제대로 빌드되면, `docker push` 명령을 이용해서 깃허브 컨테이너 레지스트리에 빌드된 이미지 푸시
  - `docker push ghcr.io/[깃랩 사용자 이름]/[패키지 이름]:1.0.0`

# GHCR을 이용한 도커 컨테이너 배포하기

- 깃허브 계정에 있는 Packages 메뉴에 올라와 있는 도커 이미지



- EC2 인스턴스에 접속해서 역시 docker login 명령으로 깃허브 레지스트리에 접속-이미지 다운받기
  - `docker pull ghcr.io/[깃랩 사용자 이름]/[패키지 이름]:1.0.0`

# EC2 인스턴스에서 AI 에이전트 프로젝트 실행하기

---

- EC2 인스턴스에 사용할 수 있도록 docker-compose.yml 수정
  - `mkdir -p ~/my_memo_app_mvc_docker`
  - `cd ~/my_memo_app_mvc_docker`
  - `nano docker-compose.yml`
  - `cat docker-compose.yml`
- 컴포즈 실행하여 배포
  - `docker compose up -d`

# EC2 인스턴스에서 AI 에이전트 프로젝트 실행하기

- 실행 화면

The screenshot shows a web browser window with the address bar displaying '주요 요약' and '2024.11.12:8000'. The page content is as follows:

**마이 메모 앱에 오신 것을 환영합니다!**

간단한 메모를 작성하고 관리할 수 있는 앱입니다.

사용자 이름:

비밀번호:

**로그인**

사용자 이름:

이메일:

비밀번호:

**회원가입**